

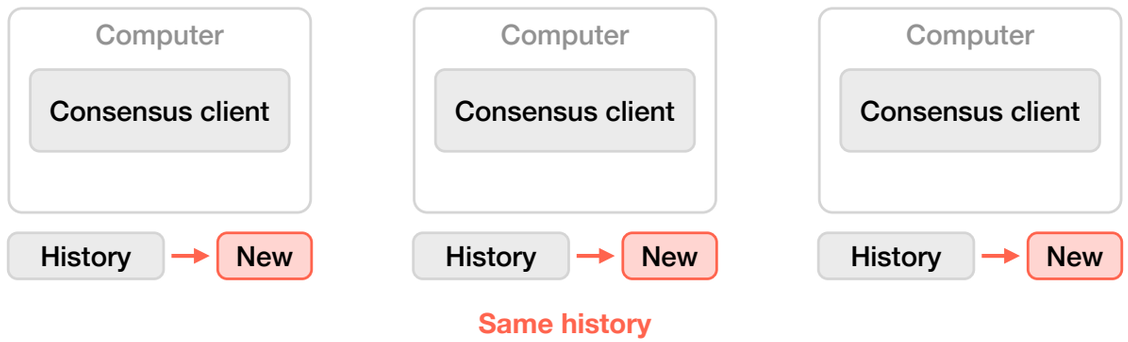
SpecTrum:
Specification-Guided Differential Fuzzing for Ethereum Consensus Clients

Seokhun Jeong[1], Gyeongmin Dan[2], Sukyoung Ryu[1], Sungjae Hwang[2]

Hello, I’m Seokhun Jeong.
Today, I will present SpecTrum, which uses Ethereum’s specification to find client disagreements that existing tests missed. This is joint work with my advisor 석영류, and 성재 황 and 경민 단 from Sungkyunkwan Univ.

Consensus determines a shared transaction history

Every client must make identical updates to the history

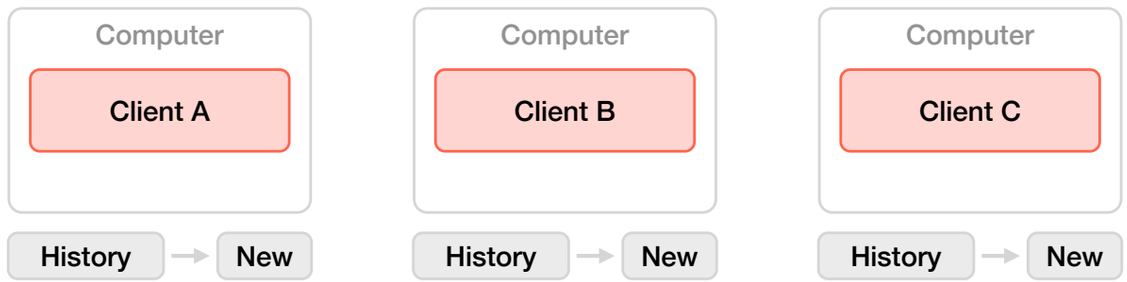


Ethereum is a network of computers that maintain a common transaction history.

Consensus is the process by which they agree on that history. Each computer runs a consensus client, a program implementing the rules for this agreement. To keep the history in sync, every client must make identical updates to the history when proposed new additions.

Client diversity reduces dependence on a single implementation

Independent implementations of the same rules

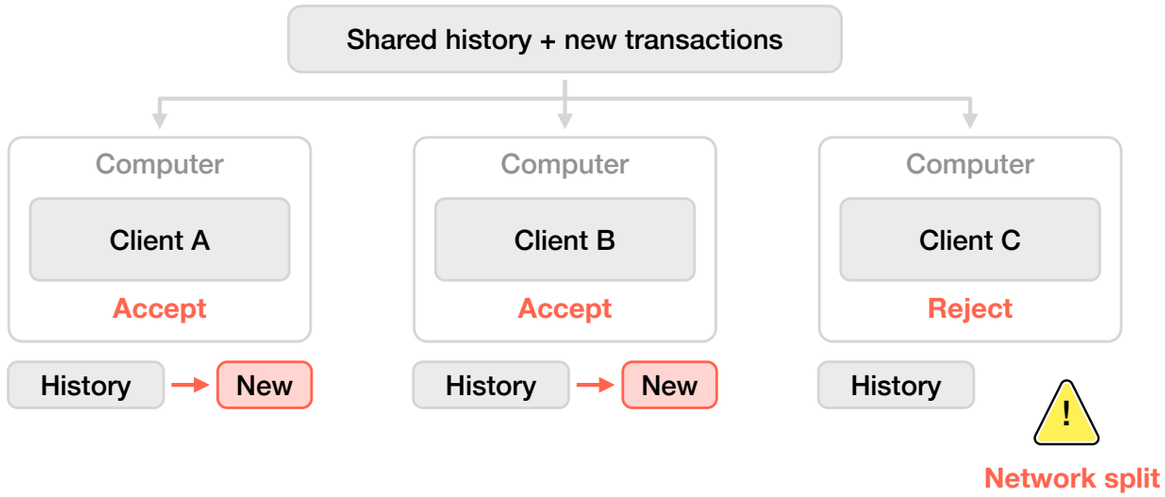


Same history

Client diversity reduces dependence on one implementation.

Ethereum uses independently developed clients, so that a defect in one client does not necessarily affect the others. However, these clients must still agree on which transactions to accept and reject.

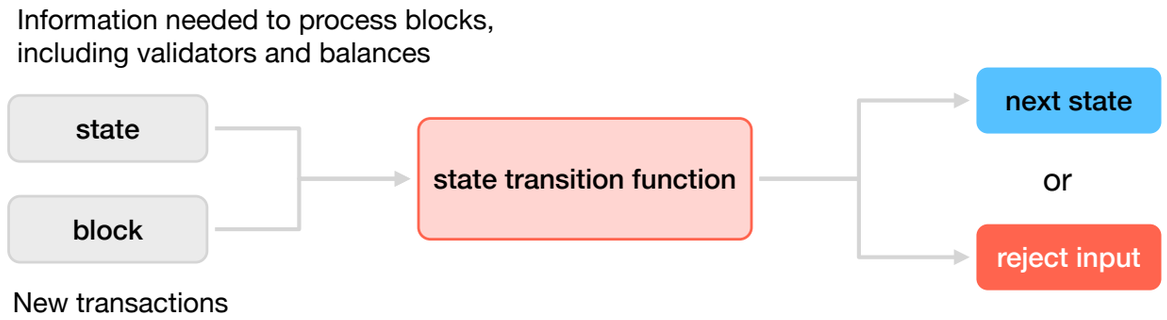
Client disagreements can split the network



Client disagreements can split the network.
Here, the same new transactions reach all three clients. Two accept it and extend their histories, while another rejects it, resulting in inconsistent history, called network split.

We look for these disagreements through differential testing: running the same input on multiple implementations and comparing their outcomes.

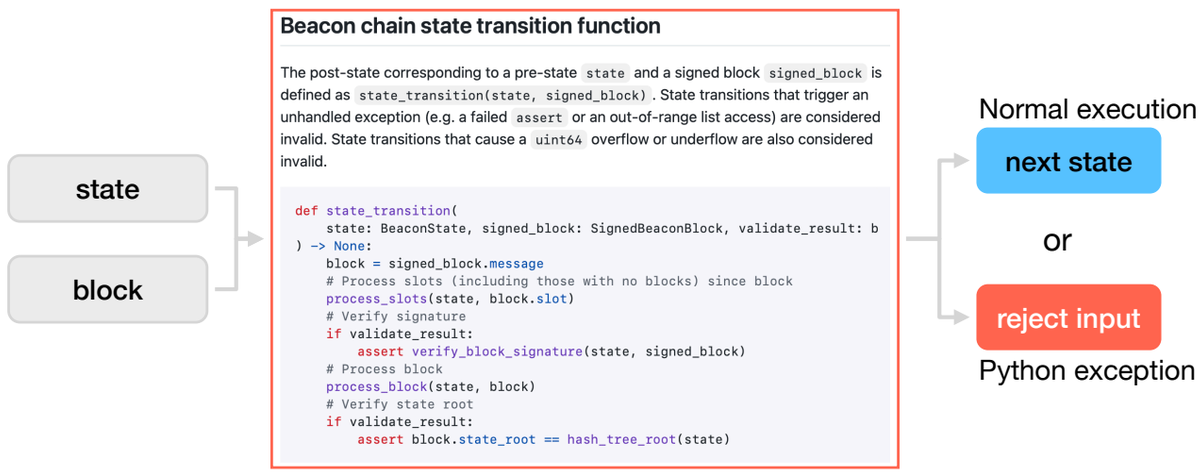
The state transition function processes new blocks



Our work focuses on the state transition function.

New transactions arrive in blocks. The state holds information needed to process a block, including consensus participants, called validators, and their balances.
The state transition function takes these two inputs and either produces the next state or rejects the input.
Clients should agree on rejection and on any resulting next state.

The official specification defines the expected outcome

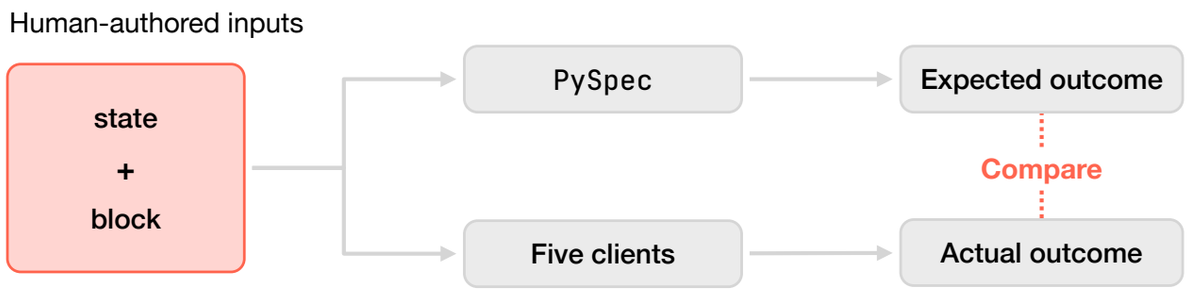


PySpec: official executable Python specification

Those expected outcomes are defined by the official specification.

The consensus specification for state transition is a Markdown document containing Python code. The Python code can be executed, and this executable form is called Pyspec. Running it produces a next state or raises an exception, indicating rejection.

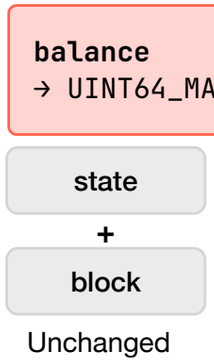
Official tests compare clients with the specification



Official tests check clients against these expected outcomes.

We target five major consensus clients, each implemented in a different language. Test authors write inputs, compute the expected outcome using Pyspec, and compare each client's results. So Pyspec defines the expected behavior, but test authors still have to decide which cases to test.

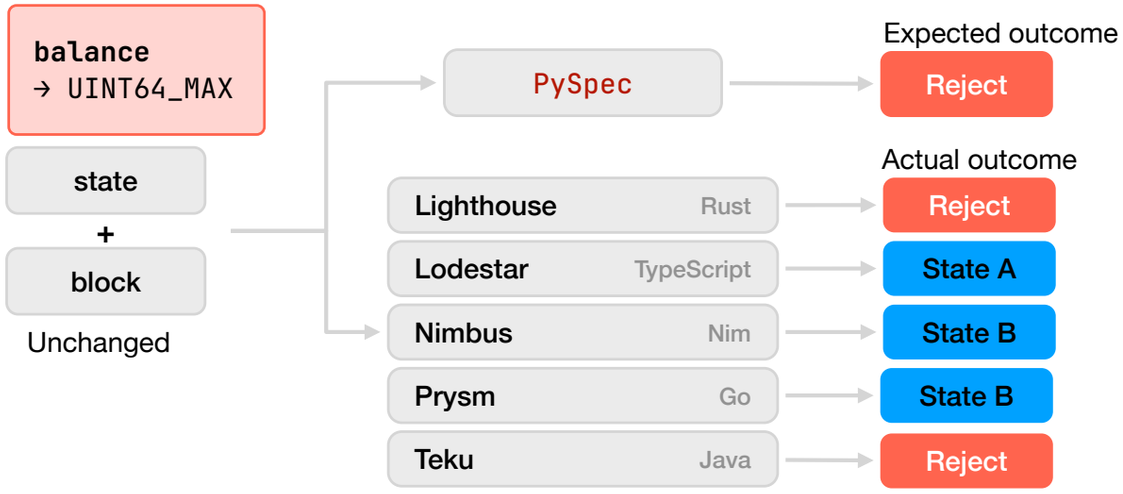
One balance mutation exposes client divergences



A small change to an official input can create a divergence the official tests miss.

Here, we change one validator balance of the input state to the maximum unsigned 64-bit value. Every other field and the original block stay unchanged.

One balance mutation exposes client divergences



This changed input made the clients disagree.

The specification’s expected outcome is to reject the input. Two clients reject it as intended, but the other three accept it and produce two different next states.
Such differences in state transition outcome are called divergences.
The required behavior was specified in the Pyspec, but the official tests did not exercise this particular case.

Tests can miss implicit exception conditions

PySpec

```
if prev_bal * 3 >= total_bal * 2:
```

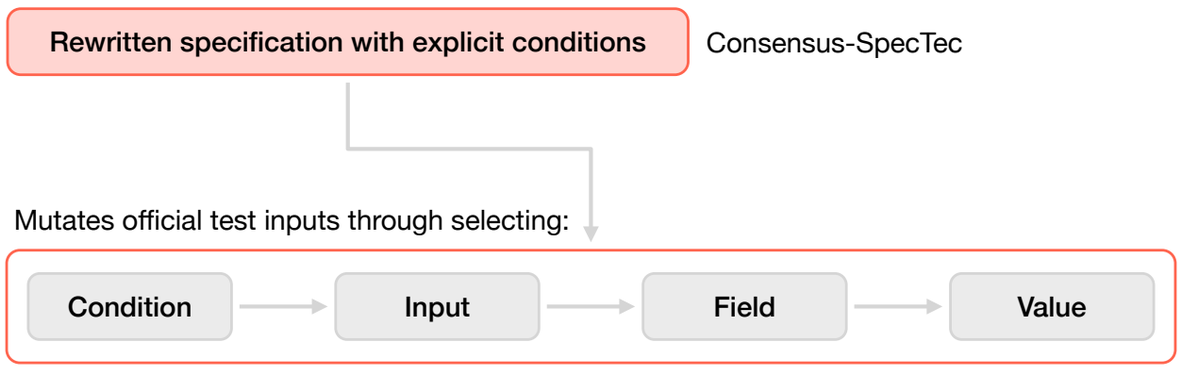
Expected outcome: uint64 overflow → Python exception → reject input

Implicit exception conditions: conditions hidden in operations or type behavior

One reason tests miss such divergences is that some exception conditions are implicit.

They follow from operations and types, without a separate check in the state-transition code.
Here, total balance is a sum of input balances, represented as an unsigned 64-bit integer. Doubling it can overflow and require rejection.

SpecTrum uses explicit conditions to guide automated test generation



Our tool SpecTrum uses explicit conditions to guide automated test generation, or fuzzing.

We first manually rewrite the specification to make conditions explicit. The rewritten specification guides mutations, or changes to existing test inputs, by selecting a target condition of the rewritten spec, an existing input, a field of the input, and a new value to replace the field.

Test generation needs conditions for each execution result

Example state update function

```
1 if P:
2   S.x = A
3 if Q:
4   S.x = B
```

S: input state
P, Q: comparisons
A, B: new values for field x

Execution result: return value and state updates

Both outcomes of each branch tested

P	Q	Execution result
true	true	`S[x ← B]`
false	false	`S`

Missing state update: `S[x ← A]`

Test generation needs conditions that distinguish execution results.

For normal execution, this includes both return values and state updates. Here we focus on state updates because our target is a state transition function.

Comparisons P and Q can update field x to A and B. Testing both true and both false exercises every branch outcome, but it misses the case where the final state gets A. Because we want to cover all possible execution results, we expose the conditions for each result.

Each execution result is paired with its path conditions

Example state update function

```
1 if P:
2   S.x = A
3 if Q:
4   S.x = B
```

Manual rewrite

Path condition: when the execution result occurs

Path condition	Execution result
¬P ∧ ¬Q	S
P ∧ ¬Q	S[x ← A]
Q	S[x ← B]

Each normal execution matches exactly one row (mutually exclusive and exhaustive)

We organize the specification around execution results. Each execution result is paired with its path condition, describing when it occurs.

Updating x to A requires P to be true and Q false. When Q is true, x ends at B regardless of P, so those paths share one row. Every normal execution matches exactly one row, meaning the path conditions are mutually exclusive and exhaustive.

Exception prevention conditions make rejection explicit

Comparison P

```
if prev_bal * 3 >= total_bal * 2:
```

Exception prevention condition

```
total_bal <= UINT64_MAX / 2
```

true	false
multiplication stays within uint64 range	overflow exception, input rejected

We also make rejection explicit through exception prevention conditions.

The comparison written as P includes the earlier multiplication of total balance by two. We guard this with an exception prevention condition, which says multiplication stays in range when the total is at most half the unsigned 64-bit maximum.
If this condition is false, the multiplication would overflow, so the input should be rejected. The rewritten specification includes both path conditions and exception prevention conditions.

The results and conditions are made executable with P4-SpecTec

One rule from Consensus-SpecTec (simplified)

```
rule WeighJustAndFin/justify_prev:
  state, total_bal, prev_bal, curr_bal
    ~> state[.CURR_JCP = prev_epoch]

-- if total_bal <= UINT64_MAX / 2
-- if prev_bal <= UINT64_MAX / 3
-- if curr_bal <= UINT64_MAX / 3

-- if prev_bal * 3 >= total_bal * 2
-- if ~(curr_bal * 3 >= total_bal * 2)
```

Execution result:
`S[x ← A]`

Exception prevention conditions

Path condition:
P ∧ ¬Q

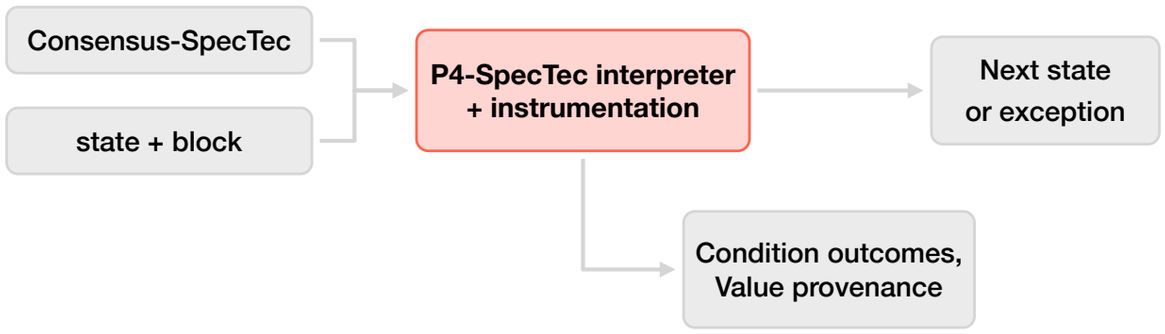
The specification is rewritten in P4-SpecTec, a specification language for executable specifications.

Each rule expresses a result together with its conditions.
This fragment shows the earlier update to A, using the original state field and epoch value. The expression after the arrow is the resulting state.

Each double dash introduces a condition. The first three are exception prevention conditions that prevent overflow. The last two form the path condition, P and not Q.

These rules collectively form the rewritten specification, called Consensus-SpecTec.

Both specifications agree on all 910 official tests



Agrees with PySpec on **910 official tests** (state transition + unit tests)

We checked that Consensus-SpecTec agrees with Pyspec on the official tests.

The P4-SpecTec interpreter executes Consensus-SpecTec, together with an input state and block pair.

Instrumentation records execution outcomes of each condition and tracks which input fields contribute to each computed value.

Across all 910 official state transition and unit tests, the two specs produced matching next states or exceptions. We use the checked Consensus-SpecTec to guide test generation.

Condition coverage identifies an untested false outcome

```
total_bal <= UINT64_MAX / 2
```

Condition outcomes on 581 **state + block** pairs

true	false
21	**0**

Selected as target condition

*Tautologies are manually excluded

Condition coverage records whether each condition evaluates to true and false.

Across 581 official state transition inputs, this condition was evaluated 21 times to true and never to false. Conditions like this one, that have been reached but never evaluated to false become target conditions. We manually exclude conditions that cannot be false.

An input that reached the condition becomes the seed

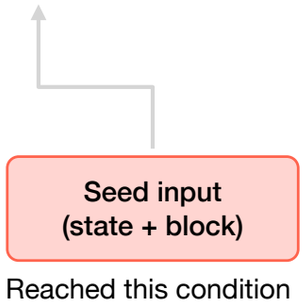
```
total_bal <= UINT64_MAX / 2
```

Condition outcomes on 581 **state + block** pairs

true	false
21	**0**

Selected as target condition

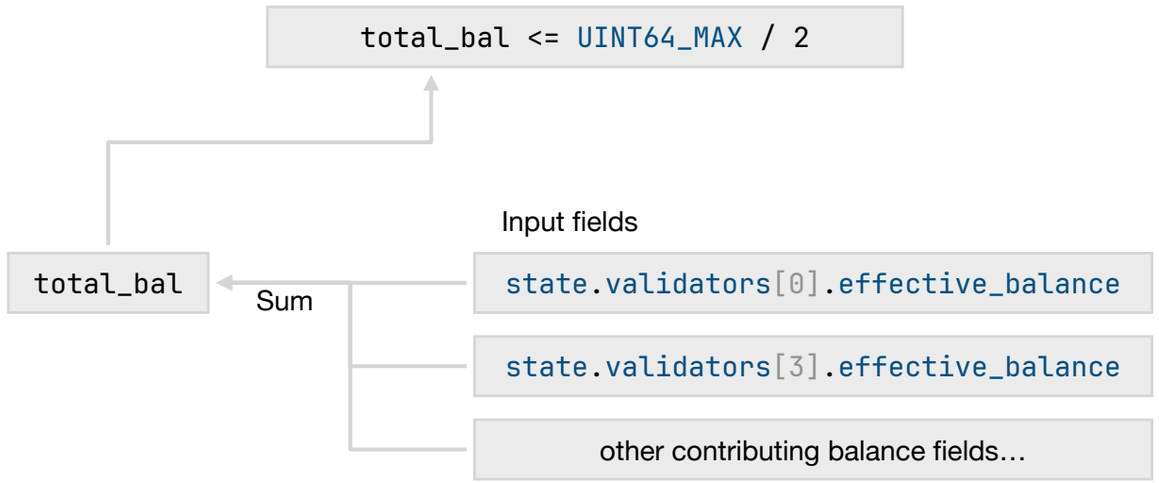
*Tautologies are manually excluded



An input that reached the target condition becomes our seed, the starting input for mutation.

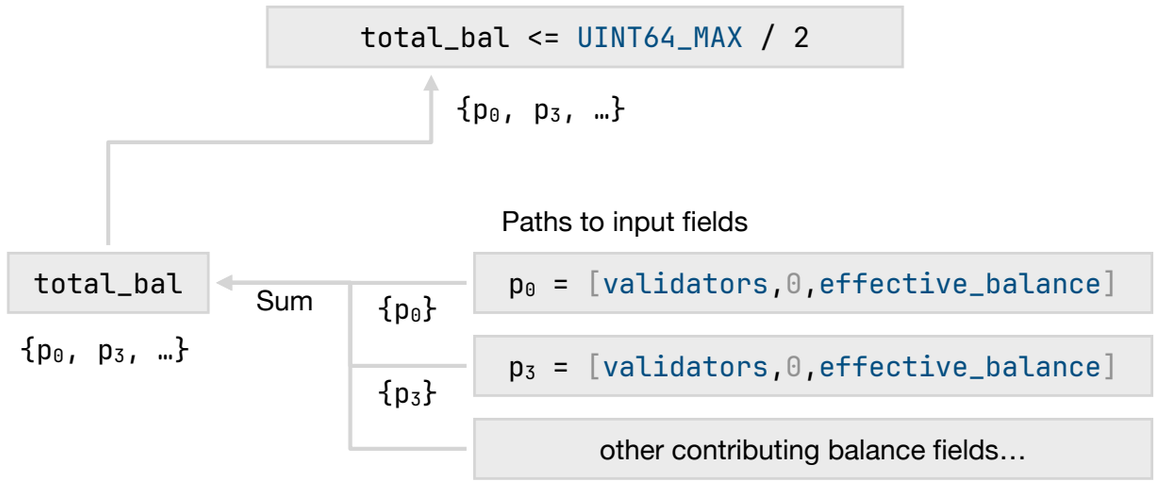
Next we identify which field of this seed input to mutate.

Provenance connects the condition to input fields



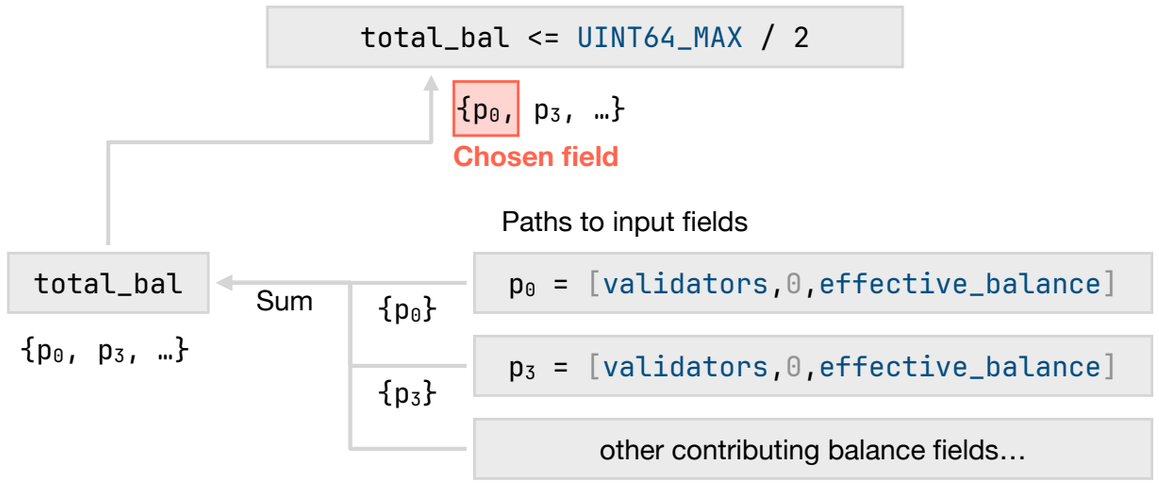
Provenance tracking records which fields of the input contributed to a computed value.
The target uses total balance, a sum of certain input balances in the state.

Provenance connects the condition to input fields



A field path identifies a location in the input. Here, p_0 identifies one validator's balance. Whenever values are combined and created during execution, their contributing field paths, or provenance, propagates through the interpreter.

Target condition's provenance identifies field to mutate

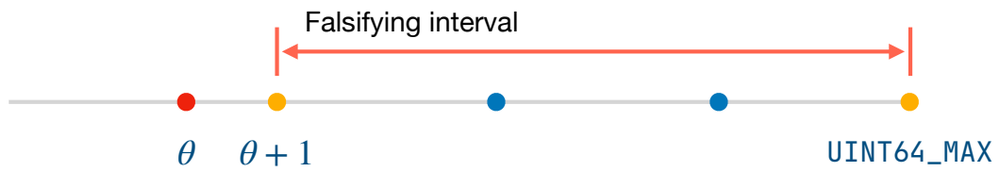


When we reach the target condition, its provenance tells us which fields contributed to the condition. We select one of these fields, p0, for mutation.

Target condition expression guides replacement values

```
total_bal <= UINT64_MAX / 2
```

$\theta = \lfloor$UINT64_MAX / 2$\rfloor$

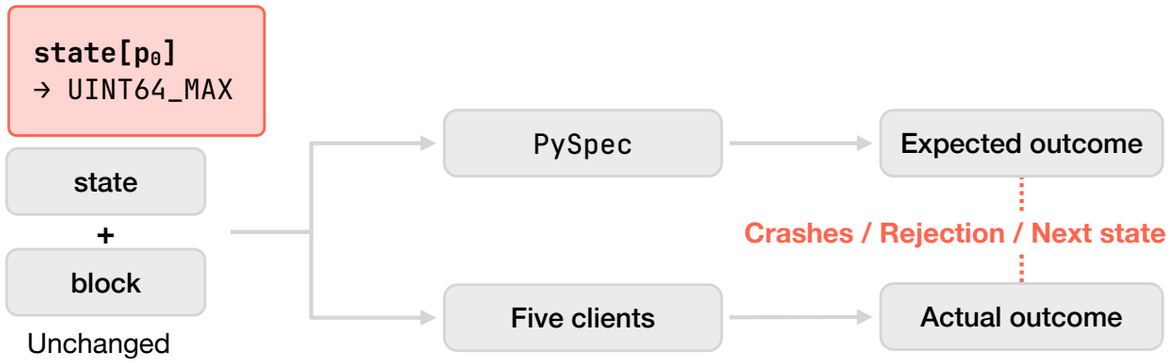


● Boundaries, ● interiors, and ● transition values of interval

Next, the condition expression is used to guide replacement values.

Calling the boundary theta, we first compute the interval that makes the condition false. Then we use its boundaries, interiors, and transition values just outside the interval. This is a heuristic; whether a mutation actually reaches the condition and makes it false depends on execution.

Differential testing compares outcomes on each input



Each mutation replaces only one selected field with one candidate value, and leaves the other fields and original block unchanged. We then run differential testing on Pyspec and the five clients. We compare crashes, rejection decisions, and the next states.

SpecTrum found 27 divergences across five clients

21,444 generated tests across five clients (Capella)
Cases: grouped by root cause and client behavior

Category	Cases	
Input constraint violations	●●●●●●●●●●●	11
Integer range handling	●●●●●●●	7
Integer decoding errors	●●●●	4
Function precondition violations	●●●	3
Protocol version selection errors	●	1
Computation mismatches	●	1
		27

23
block production only

4
also in block validation

The generated tests revealed 27 divergence cases across six root cause categories.
Cases group inputs by root cause and client behavior pattern.
Twenty-three appeared only in block production, when clients create and propose blocks. Four also appeared in validation, when clients check received blocks.

Generated tests made 126 additional conditions false

Measurement	Official tests	With generated tests	Increase
Conditions evaluated to false	[1] 130/281 (46.3%)	256/281 (91.1%)	+126
`PySpec` branches covered	152/358	154/358	+2

[1] Denominator excludes conditions that cannot be false

Generated tests made 126 additional conditions false, increasing the proportion from about 46 to 91 percent after excluding conditions that cannot be false. Pyspec gained only two covered branches. Implicit exceptions do not have separate branches in the state-transition code, so explicit conditions reveal gaps in tests that branch coverage struggles to detect.

Explicit conditions expose new divergences

65 of 126
Newly falsified conditions

Exception prevention conditions
added during rewrite

22 of 27
Divergence cases

Found by targeting
exception prevention conditions

Newly explicit conditions helped exercise exceptions and find divergences.
Of the 126 newly falsified conditions, 65 were exception prevention conditions introduced by the rewrite. Also, 22 of the 27 divergences were found by targeting such conditions.

SpecTrum found more divergences with the same test count

21,444 tests generated by each method

Method	Field and value selection	Cases
Random	Random field and random value	12
Extreme	Random field and boundary value	16
SpecTrum (this work)	Specification conditions and value provenance	27

Seeds: baselines 581, SpecTrum 59
Random and Extreme each found five cases that SpecTrum missed

SpecTrum found more cases than baselines at the same test count.

Random chooses a random field and value. Extreme uses a type boundary value instead. With 21,444 tests each, they found 12 and 16 cases, versus 27 for SpecTrum. Each baseline also found five cases SpecTrum missed.

Limitations

Search scope

Selects 59 seeds from 581 inputs. Does not target conditions already evaluated as false.

Manual effort

Rewrite the specification and exclude conditions that cannot be false.

Live network impact

Input reachability and exploitability remain unverified.

Our search scope, manual preparation, and unverified network impact limit the approach.

SpecTrum used 59 of the 581 official inputs as seeds and did not target conditions already evaluated as false. Preparing the specification and excluding infeasible targets also require manual work. Finally, we have not established whether the generated inputs can arise on the live network or be exploited.

Explicit specification conditions reveal testing gaps

Expose path and exception conditions

↓

Measure condition coverage to select targets

↓

Track value provenance to guide mutations

+126

conditions evaluated as false

27

divergence cases across five clients

The takeaway is that making path and exception conditions explicit reveals gaps in existing tests.

Condition coverage identifies targets, and value provenance guides mutations. This approach made 126 additional conditions evaluate to false and revealed 27 divergence cases across five clients. Thank you for your time and attention.